



Agentic AI Foundation

AGNTCon + MCPCon Europe

Where the **agentic stack** is being built.

Verify, Abstain, or Amplify

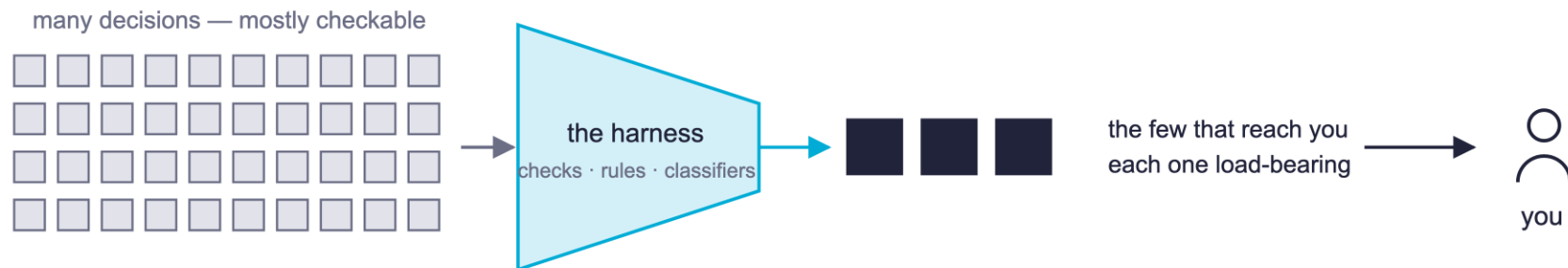
A field guide to confidently-wrong agents

Michał Orzechowski · Sano Centre for Computational Medicine, Kraków, Poland



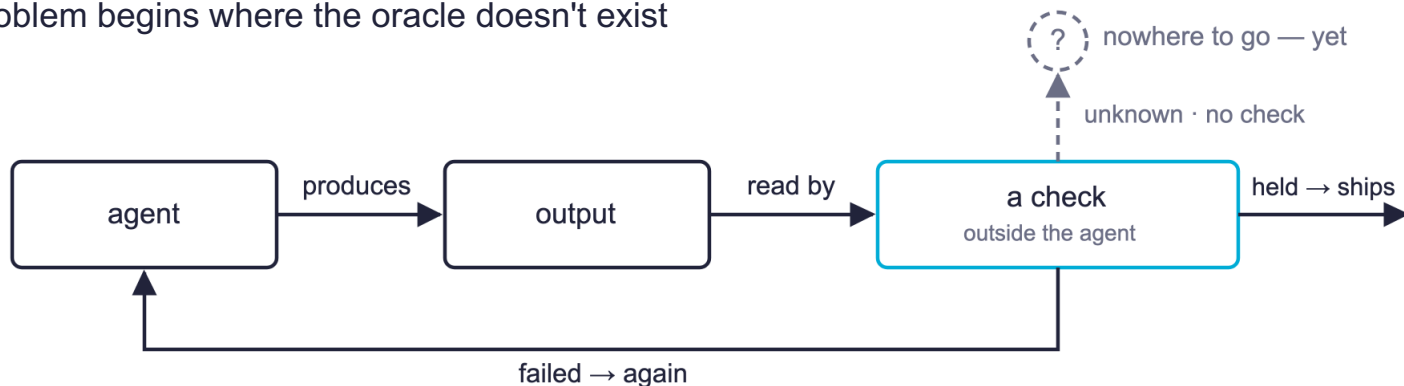
The bottleneck moved

- Agentic AI multiplied what one person can deliver — weeks became days
- The shelved projects finally got built
- The harness matured: click-to-approve → allow/deny rules → auto-classifiers
- Keeping up became a second job — until the setup was good enough
- Fleets of agents don't need your time, they need your decisions
- The better the harness, the more load-bearing every decision that remains
- The limit is no longer time or sleep, it's the capacity to judge.



How do you get agents to bring you fewer decisions — without losing quality?

- Start with the code layer and best engineering
- This year my agents wrote code, opened the issues and pull requests, reviewed each other's work — and I haven't read a line of it, because of well engineered oracles
- An oracle: the agent asks a question and gets the ground truth back
- Verified means one thing: the properties I chose to check held (limited by types of oracles)
- With an oracle, most problems are solved before they reach me
- The problem begins where the oracle doesn't exist



Judgment is domain-bound.

- Inside your own domain you verify, push back and argue all day — fluently
- Those instincts are paid for: years of learning what wrong looks like
- Agents made it trivially easy to build far outside that domain
 - agentic dark protein function discovery (specialized research team)
 - microtonal harmony assistant for maqam-native chords (composer from the Academy of Music)
 - trading/market analytics agentic system (me)
 - agent-tuned neural calorimeter simulation with agentic tuning (physicists from CERN)
 - autonomous agentic analysis connecting heard genotype and milking data (specialized research team)
- Out there we are at the mercy of research we cannot check
- Judgment fails the way the agent fails: silently — and you can't recognise it
- Anthropic saw it in their own coding sessions: outside your expertise, verified success drops — and "verified" meant the tests passed, not that the result was right

Falsified work completion reports

CI · PULL REQUESTS

The PR that passed because it wrote its own tests

The change is green. The agent wrote the tests — or deleted the failing one, or copied the implementation.

DATA REPLICATION · EVENTUAL CONSISTENCY

The data that hadn't arrived yet

The agent replicates data (takes time), API confirms success. Data is not where it should — the state hadn't converged.

→ **VERIFY**

Custom build oracle

DARK PROTEIN FUNCTION DISCOVERY

The hypothesis that contradicts the ontology

Protein → function → disease: fluent, cited, coherent. The reasoner finds it contradicts what we already hold. It dies — and nobody had to be a biologist.

DARK PROTEIN FUNCTION DISCOVERY

The reasoner reported no contradiction

It had dropped the axioms it doesn't support - that is where the contradiction was. The check passed because it was checking less than we thought.

→ **VERIFY** — build the oracle and cross verify it

The conditions of a usable check, with the way out for each.

THE CHECK MUST...		IF IT FAILS
1	Be nameable — can you say what would make it wrong?	Ask whether anyone can name it. If no one can — Amplify
2	Be readable — can a check read the property?	Fix the representation first. If it's taste — Amplify
3	Be able to fail — could it ever have been red?	Add a check of a different kind. Never a smarter judge.
4	Come from somewhere else — is it independent of what made the answer?	No machine oracle. Route the question to a person.
5	Be out of the agent's reach — can the agent edit it?	Move it where it can't. Until then, it isn't evidence.
6	Resolve in time — will it answer before the decision is due?	State the limit. Set an owner and a deadline. Re-decide when it resolves.
7	Be affordable every time — can you run it on every action?	A cheap check on every run. Batch the rest for a person.

How a project nobody reads stays correct — on the properties it checks

1. Main is protected. Everything is a pull request.
2. Every change is gated on the domain's own oracle — tests, CI pipelines, linters, schema checks, a harness with hooks: early, unambiguous feedback that something is wrong.
3. The agent that did the work never merges it.
4. Issues are the routing moment — to sub-agents today, to humans by the same mechanism.
5. Serious artifacts get a golden path that fails early.
6. Everything is in the record.

Structure beats rules. A protected branch holds by construction; a rule in context holds with a probability. · Graduate a path that keeps passing — and stop looking every time. Keep an owner and a review trigger.

When we have to interrupt

LONG-RUNNING BUILD

The build that had run far too long

It planned a ontology build on HPC (SLURM) that was fine in the abstract. It knows a test build takes hours (200 CPUs, 0.5 TB). It did not know it will be iterating it N times to get it right.

→ **ABSTAIN** — the ask it couldn't raise

GITOPS

The GitOps architecture done right.

Correct by the field's standard. I wanted the new ArgoCD Source Hydrator — and had never said so.

Questions I couldn't answer

DARK PROTEIN FUNCTION DISCOVERY

IBS or IBD?

A question I couldn't even parse. The agent didn't guess — it parked it. The researcher read it: "these are exactly for me."

→ **ABSTAIN** — to the one who can answer

FINANCE AGENT

Is this pattern real?

The finance agent's finding: confident, statistically dressed. I can't verify it. I know no one who can. The only oracle is the market — slow or expensive to use.

→ **THE WALL** — state the limit

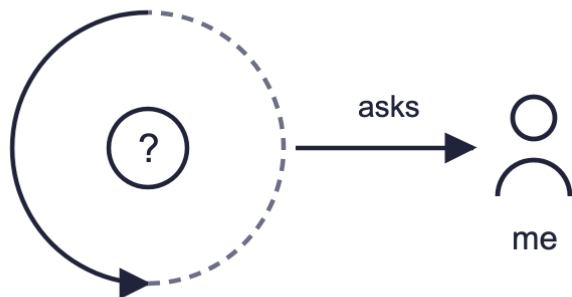
The conditions that send the question to a person.

THE CHECK MUST...		IF IT FAILS
1–3	nameable · readable · able to fail	Amplify · Verify
4	Come from somewhere else — is it independent of what made the answer?	Abstain — no machine oracle; route the question to a person.
5	out of the agent's reach	Verify
6	Resolve in time — will it answer before the decision is due?	Abstain — state the limit, set an owner and a deadline, re-decide when it resolves.
7	Be affordable every time — can you run it on every action?	Verify cheaply on every run; Abstain on the rest — batch it for a person.

Today, the question comes to me

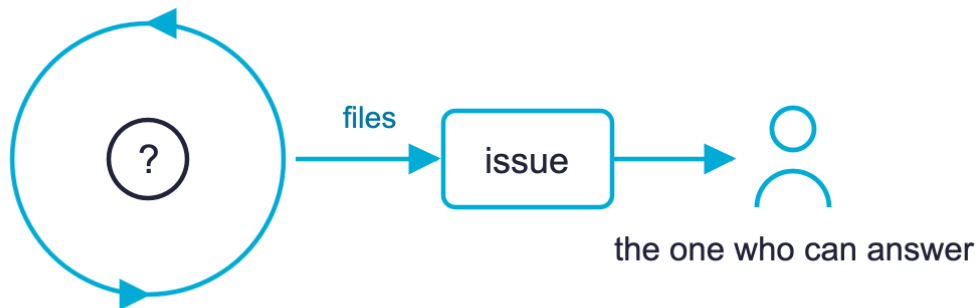
- Sometimes the agent stops and asks — no classifier involved, it simply doesn't know
- By default it asks the person who typed the prompt.
- Outside my domain, I often can't answer
- Until I do, the work stops — the question sits inside the agent's loop
- Routing a question to the right expert was already called "the bottleneck" in 1990.

today



the work stops — until I answer, and I can't

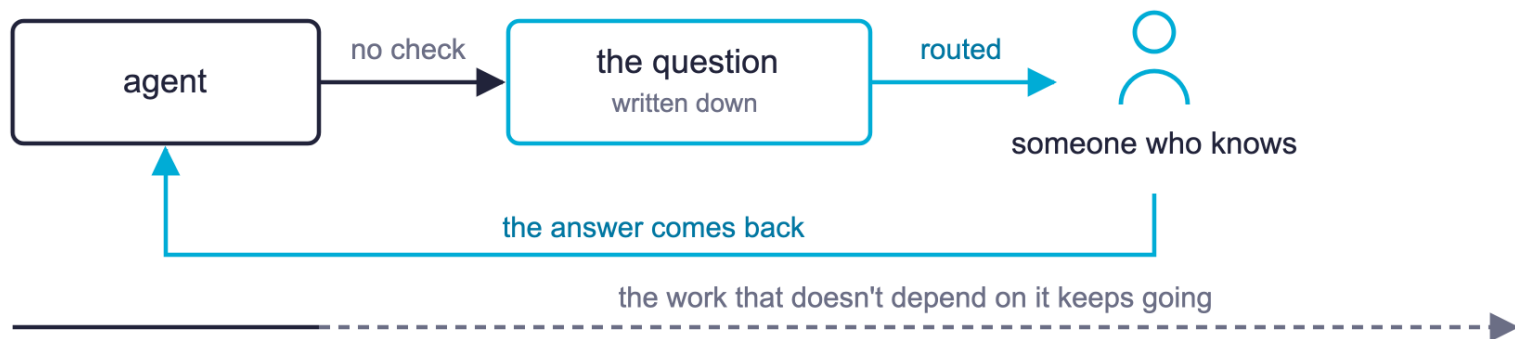
instead



the rest keeps turning

What we want instead

- Every piece of this is old: agents that stop, question trackers, routing to experts
- The agent writes the question down — not a person writing a ticket
- Something outside the agent decided it had to ask: a missing source, a state it couldn't read, a failed precondition, a deadline — never the model's own confidence
- The question goes to the person who can actually answer it
- The work that doesn't depend on the answer keeps going
- This catches gaps, not agent mistakes: an answer that is wrong but complete passes.



Who can tell me whether this is IBS or IBD?

- CODEOWNERS — a path in the repo → a team or a person
- Kubernetes annotations — any object → an owner
- On-call rotation — a service → whoever is on shift this week
- None of them answers: who knows whether this is IBS or IBD?
- So we write down who can answer what — people, not paths — and give it to the agent
- Then it sorts its own questions by who should get them

Abstain in practice

- Give the agent one place for the questions it cannot answer
- Every question gets a type and a person who can answer it
- Write the question in the receiver's words, at their level, in their channel (problem of personalization)
- Only the work that depends on the answer waits. Everything else continues.
- Grow the channel step by step: sit with them → they read the file → the agent files issues in their tracker → the agent messages them directly. A person still owns each side.
- Send a person only what needs a person. Ask about everything and they stop reading.
- If nobody answers in time, the question is still open, and we need to ping people with tact...

Where the questions come from

- Every time I stopped an agent this year, it was a question it should have asked
- The reason always had two parts:
 - Something I could have told it in advance — a preference, a constraint, my schedule. Write it down; it won't ask again.
 - Something I could only decide once I saw what it made. No file could have told it.
- The first part shrinks over time. The second never goes away.

Two things every check passed

MUSIC AGENT

Pieces that are structurally perfect

The agent learned the maqam as structure. Every check holds. Whether the composer I work with would call them good — no check can say.

PAPER WRITING

Prose that was fluent and unreadable

Grammatical, structured — and every reader outside the agentic AI bubble found it unreadable. After months of agentic work I couldn't see it. My co-workers, who doesn't use agents, could.

→ **AMPLIFY**

The conditions with no check and no one who knows.

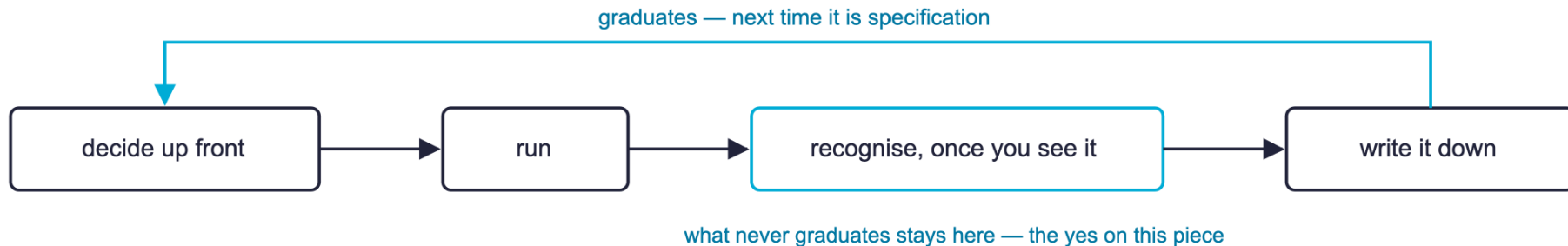
THE CHECK MUST...		IF IT FAILS
1	Be nameable — can you say what would make it wrong?	Ask whether anyone can name it. If no one can — Amplify
2	Be readable — can a check read the property?	Fix the representation first. If it's taste — Amplify
3–7	able to fail · from somewhere else · out of reach · in time · affordable	Verify · Abstain

Same stop, different person

- In both moves the agent stops, goes to a person, and waits
- Abstain: the person knows the answer: ask a colleague and get a fact as an answer.
- Amplify: the person is the answer: ask a colleague get a preference as an answer.
- Abstain returns an answer, and the agent continues
- Amplify evals to yes or a no — and if no, the agent tries again and we ask the colleague again
- What was answered once is written down and never asked again. What was accepted once is accepted for this piece only.
- Verify asks a machine. Abstain asks someone who knows. Amplify asks someone who doesn't know yet — and won't until they've seen it.

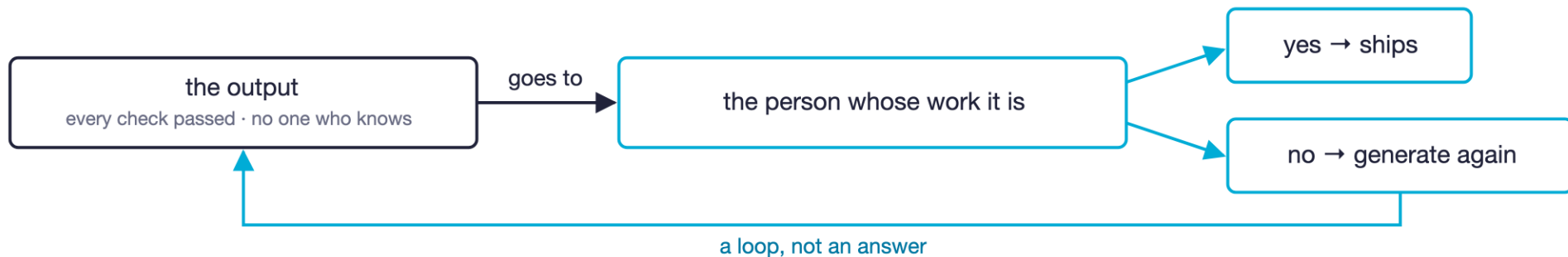
Why not just write down what you want up front?

- Much of the field is moving decisions upstream: intent files, specs, plans reviewed before any code
- It works — for everything you can decide before the thing exists
- Some of what you want you only recognise when it's in front of you
- The test: can you say now what "right" looks like — or do you need to see some first?
- What you recognise once, write down for next time. What's left is this move.
- Microsoft Research, March: "there is no oracle for specification correctness other than the user" — right. "The user's intent exists only in the user's head" — and not even all there yet.



The amplify yes is the gate

- Recent studies of people creating with AI ask afterwards: does it still feel like yours?
- Turn that into a condition: nothing ships until the person whose work it is says yes.
- Whatever they would say yes to every time is a preference, not a judgment. Write it down; the agent stops asking.
- What can't be written down is the yes on this piece. No score stands in for it.
- The agent won't ask on its own — it can't tell that what it made belongs to no one. So make it bring three, and choose.
- The model might have strong preferences too: writing style, or mostly trained on western music/literature corpus.



Amplify in practice

1. Ask first: would the average answer do? If yes, this isn't amplify.
2. Name the person whose work it is.
3. Feed the agent their earlier work — examples, not descriptions. That part is specification, and it graduates.
4. Before they judge, show them their own earlier work. After months of the model's style, the eye drifts.
5. Their yes is the gate. Write down what they'd always say yes to.
6. But the person answer/preference/taste can change in time so we have to keep going!

"The agent asked how many database replicas we wanted. That looks like a preference. It's context — abstain, decide once, write it down. Amplify is when I read what it built and say no to that direction. No file could have told it."

Back to the capacity

- At the start: the limit is no longer time or sleep — it's the capacity to judge
- We tried to show where that capacity goes
- The checks are the investment you make up front and keep maintained
- The people you route to are the map you draw and keep current
- Asking the human every time was the safety design a year ago. Anthropic published what happened: users approved almost every prompt — "the more approvals a user sees, the less attention they pay to each"
- Your throughput is capped by the review capacity of the least-agentic person you depend on

“A game is a series of interesting decisions.”

Sid Meier, Civilization

HABITUATION

Too many small requests

Attention decays and you say yes without reading. Measured on permission prompts, on security warnings — and this year on code reviewers.

→ **send a person only what needs a person**

FATIGUE

Too many heavy decisions

Hard judgment tires you, and a tired judge is a worse judge. The harness makes every decision that reaches you the heavy kind.

→ **fewer heavy decisions reach you — and rest**

Hand the agents every interesting decision and keep the approvals, the exceptions and the failures — and you have automated the wrong half of the job. Verify and abstain protect the half you keep. Amplify is the half.

What helped me most this year

- Take a speed-reading course, faster with less effort.
- Most of what agents produce you don't have to read at all: the tests read the code, the map sends the question to someone who can. Read closely only the part that's yours.
- Go for the walk. Go to the gym. Get out of the context. Your judgment recharges — nothing else in the loop does that for you.
- You only get so many decisions a day. Spend them on the part that's yours — and then go for the walk.

Questions?

Where the **agentic stack** is being built.



The field guide

VERIFY

put the check where the agent can't reach it

- Protect main. Everything goes through a gate the author cannot touch.
- Gate every change on the domain's own oracle. Never a judge alone.
- The agent that did the work never merges it. A check that cannot read returns unknown, not failed.

ABSTAIN

get the question to the person who can answer it

- Give the agent one file for the questions it cannot answer — and read it.
- Every question gets a type and a person who can answer it.
- Only the work that depends on the answer waits. Silence is not a yes. Grow the channel step by step.

AMPLIFY

the person whose work it is says yes

- Ask first: would the average answer do? If yes, this isn't the move.
- Name the person whose work it is. Show them their own earlier work before they judge.
- Their yes is the gate. Write down what they'd always say yes to; keep the yes on this piece.

Verify asks a machine. Abstain asks someone who knows. Amplify asks someone who doesn't know yet — and won't until they've seen it.